

ML-Based Decision Support System for Smart Investments

Dr. Reshma Koli¹, Avanish Vadke², Atharva Shelke³,
Abhishek Thormothe⁴, Aadit Suryawanshi⁵ and Prof. Harshada Sonkamble⁶

¹²³⁴⁵⁶A.P. Shah Institute of Technology, Thane, India
rrkoli@apsit.edu.in, avanishvadke3@apsit.edu.in,
atharvashelke2@apsit.edu.in, abhishekthormothe150@apsit.edu.in,
aaditsuryawanshi60@apsit.edu.in, hksonkamble@apsit.edu.in

Abstract. Retail investors and discretionary traders make decisions under uncertainty, limited information, and behavioral bias; without a measurable workflow, these decisions can lead to avoidable drawdowns. This paper presents *QuantFin*, a repository-implemented, full-stack decision support system for Indian large-cap equities (Nifty 50 constituents available in the dataset). QuantFin integrates (i) robust ingestion of OHLCV time series from multi-header CSV exports; (ii) a technical-indicator feature pipeline; (iii) a multi-model prediction suite spanning regression and classification (ridge regression, logistic regression, SVM, an ARIMA baseline, and LSTM); (iv) portfolio construction via ML-driven ranking and optimization-based baselines; and (v) leakage-aware walk-forward backtesting that evaluates rebalanced strategies against a benchmark. The emphasis of this paper is **truth-only, reproducible reporting**: all quantitative values stated are taken from saved repository artifacts and the implemented backtest engine. In the current repository state, the backend dataset contains 49 usable per-symbol CSV files spanning 1991-01-02 to 2025-10-06. For example, saved training logs for RELIANCE report test accuracy 0.5003 (logistic regression) and 0.5136 (SVM) for directional classification, and LSTM test MAPE 20.447% for price forecasting. In a reproducible walk-forward run (monthly rebalancing, 2023-01-01 to 2024-01-01), the implemented LINEAR strategy achieved CAGR 48.31% versus 30.60% for the benchmark under the repository’s assumptions.

Keywords: Decision Support Systems, Quantitative Finance, Walk-Forward Backtesting, Time-Series Forecasting, FastAPI, Portfolio Construction

1 Introduction

Quantitative investing systems increasingly combine statistical modeling, machine learning, and portfolio construction, but reported performance is highly sensitive to evaluation protocol. In particular, time order must be respected, and realistic constraints (e.g., rebalance cadence, capital budget) must be explicit. Recent work motivates integrating prediction with portfolio decisions while maintaining evaluation discipline [1, 3].

QuantFin is designed as an end-to-end platform that connects these components for an Indian equity universe. Unlike isolated notebooks, QuantFin operationalizes a repeatable workflow: ingestion of OHLCV data; feature engineering; model training and inference across multiple approaches; portfolio construction and rebalancing; and walk-forward backtesting. The implementation consists of a FastAPI backend (data + ML + portfolio services) and a React/TypeScript dashboard.

Contributions. This work contributes:

- A repository-implemented architecture for interactive quantitative analysis (FastAPI backend + React/Vite frontend).
- A leakage-aware walk-forward backtest engine that enforces date cutoffs at each rebalance date.
- A multi-model baseline suite (linear/logistic/SVM/LSTM and an ARIMA baseline in the codebase) and multiple portfolio baselines (ML ranking, model-weighted, mean–variance, risk parity).
- A reproducibility-first reporting approach in which claimed results are anchored to saved JSON artifacts and generated figures in the repository.

Scope and non-claims. QuantFin is a prototype research system. The repository includes a validation report indicating the system is not production-ready at the time of writing; therefore, this paper avoids operational claims (e.g., deployment reliability) and focuses on implemented methodology and reproducible outputs.

2 Related Work

QuantFin builds on themes from (i) end-to-end ML decision support, (ii) stock ranking and recommendation, (iii) classical vs deep time-series baselines, and (iv) portfolio optimization.

End-to-end portfolio decision support. Aithal *et al.* [1] describe integrating prediction with portfolio management. QuantFin adopts this framing but emphasizes explicit walk-forward evaluation.

Stock ranking and recommendation. Ranking-based selection is a practical decision interface for portfolios [7]. QuantFin similarly converts predictions into per-asset scores and selects top- K candidates at each rebalance date.

Classical vs deep baselines. Comparative baselines including ARIMA and deep sequence models are common in financial forecasting [2, 4]. QuantFin exposes both families within one pipeline.

Optimization baselines. Reviews emphasize the role of covariance estimation and constraints in mean–variance and related methods [3]. QuantFin includes optimization-based baselines, while explicitly documenting simplifying assumptions where they exist in the repository.

Rebalancing and learning-based policies. Reinforcement learning approaches motivate periodic rebalancing and sequential decision framing [5, 6]. QuantFin does not implement RL, but adopts a rebalance-schedule evaluation protocol.

3 System Overview

QuantFin is a full-stack system with three layers:

- **Data + preprocessing layer:** loads per-symbol OHLCV CSV files and normalizes heterogeneous headers.
- **Model + decision layer:** computes technical indicators, trains multiple model families, and converts predictions into allocations using ranking and optimization baselines.
- **Serving + UI layer:** exposes FastAPI endpoints consumed by a React/TypeScript dashboard for interactive analysis and backtesting.

3.1 Backend services and APIs

The backend is organized around services that power REST endpoints. The repository exposes functional surfaces for (i) analytics and training summaries, (ii) prediction and portfolio recommendation, (iii) walk-forward backtesting, and (iv) portfolio reporting. The paper focuses on the behavior of these services as implemented, rather than presenting a hypothetical API.

3.2 API surface (selected endpoints)

Table 1 lists representative endpoints drawn from the repository routers. This is not intended to be exhaustive; rather, it documents the implemented interface that supports training, inference, portfolio operations, and backtesting.

Table 1: Selected API endpoints (from implemented routers).

Router	Prefix	Examples
Analytics	/api/analytics	POST /models/train, GET
		/models/training-status, GET
Portfolio	/api/portfolio	/candlestick/{symbol}, GET /kpis
		GET /summary, GET /performance, POST /rebalance, GET /strategy-comparison
Predictions	/api/predictions	GET /symbols, POST /predict, POST /portfolio, GET /model/{symbol}/{model_type}
Backtest	/api/backtest	POST /run, GET /health

3.3 Artifacts and reproducibility

QuantFin is accompanied by machine-readable artifacts (JSON summaries, cached results) that enable reproducible reporting. This paper references:

- Dataset summary describing symbol coverage and date ranges.
- Saved training summaries containing per-model metrics.
- Backtest summaries containing risk/return metrics and series outputs.
- Figures generated by repository scripts (equity curve and drawdown).

4 Data and Preprocessing

QuantFin operates on daily OHLCV time series stored as CSV files (one file per symbol). In the current repository state, there are 49 usable per-symbol CSV files. The global date range across these files is 1991-01-02 to 2025-10-06 (as reported by the repository’s dataset summary artifact).

4.1 Multi-header CSV normalization

Market data exports frequently include metadata rows or multi-row headers. QuantFin addresses this by skipping non-data rows and coercing values into a canonical schema with columns **Date**, **Open**, **High**, **Low**, **Close**, **Volume**. The normalized result is sorted by date and cached to reduce repeated I/O.

Algorithm 1 Multi-header OHLCV normalization (repo-aligned)

Require: Raw CSV file f for a symbol

Ensure: Clean dataframe with **Date**, **Open**, **High**, **Low**, **Close**, **Volume**

- 1: Read f while skipping a fixed number of header rows (export-dependent)
 - 2: Parse the first column as **Date**; drop rows with invalid dates
 - 3: Coerce numeric columns (Open/High/Low/Close/Volume); drop invalid rows
 - 4: Sort by **Date** ascending and de-duplicate as needed
 - 5: Return normalized dataframe
-

4.2 Dataset summary

Table 2 summarizes the dataset scope as captured in repository artifacts.

Table 2: Dataset scope (from repository dataset summary artifact).

Number of usable symbols	49
Global start date	1991-01-02
Global end date	2025-10-06
Data type	Daily OHLCV per symbol

5 Feature Engineering

QuantFin implements a technical-indicator pipeline that produces both return-based features and common indicators used in technical analysis. Given close price P_t , simple and log returns are defined as:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \quad \ell_t = \log \left(\frac{P_t}{P_{t-1}} \right). \quad (1)$$

Features include moving averages (SMA/EMA), MACD, RSI, Bollinger bands, rolling volatility, momentum, lagged returns, and volume-derived features. The goal is to provide a consistent feature matrix for classical models and a sequence representation for LSTM.

6 Model Suite

QuantFin includes multiple baselines to support comparison and ablation.

6.1 Ridge regression (price/return regression)

Ridge regression is used as a regularized linear baseline for regression targets. Regularization controls overfitting on correlated technical indicators.

6.2 Logistic regression (direction classification)

Logistic regression is used as a baseline classifier for up/down movement. Besides accuracy, saved artifacts include precision, recall, F1, and AUC.

6.3 Support vector machines (SVM)

An RBF-kernel SVM baseline is used for direction classification with hyperparameter selection recorded in artifacts.

6.4 ARIMA baseline (classical time-series)

The repository codebase includes an ARIMA baseline to provide a classical univariate comparator. Where metrics are not present in saved training summaries, this paper does not report ARIMA numbers.

6.5 LSTM (sequence forecasting)

An LSTM model is used for sequence forecasting with a fixed sequence length and a limited number of epochs in the stored run. Saved artifacts include regression metrics (MSE/RMSE/MAE/ R^2) and MAPE.

6.6 Repo-grounded training example (RELIANCE)

To illustrate the repository’s stored outputs, Table 3 summarizes metrics for RELIANCE taken directly from the saved training summary artifact.

Table 3: Saved training metrics for RELIANCE (from repository artifacts).

Model	Task	Test metric(s)
Ridge (linear_reg)	Regression	MAE ≈ 0.0742 ; $R^2 \approx -0.335$
LogReg	Classification	Accuracy ≈ 0.5003 ; AUC ≈ 0.5073
SVM (RBF)	Classification	Accuracy ≈ 0.5136 ; AUC ≈ 0.5051
LSTM	Forecasting	MAPE $\approx 20.447\%$

Table 4: Stored linear regression results across symbols (split date 2023-01-01; from repository artifact).

Symbol	Train samples	Test samples	Test MAE	Test R^2
RELIANCE	6768	662	0.04936	-0.13660
TCS	4778	662	0.04899	-0.13379
INFY	6771	662	0.05707	-0.01326

6.7 Additional stored regression baselines across symbols

In addition to the per-model training summary for RELIANCE, the repository contains a separate training-results artifact capturing linear regression baselines for multiple symbols under a fixed train/test split date. Table 4 reports the stored values for three symbols (RELIANCE, TCS, INFY), including sample counts and test-set regression metrics.

7 Decision Layer: Portfolio Construction

QuantFin converts model outputs into portfolio decisions using (i) ranking-based selection and (ii) optimization-based baselines.

7.1 Ranking-based selection and allocation

At each rebalance date, the system computes a per-asset score derived from predicted return and a confidence signal. Assets are filtered to remove non-positive expected returns, and the top- K are selected; weights are assigned equally among selected assets. This design keeps the decision rule transparent and makes back-test comparisons interpretable.

7.2 Optimization baselines

QuantFin includes mean-variance and risk-parity style baselines. Where applicable, the repository uses covariance estimated from historical returns in the backtest engine. Separately, the online prediction service includes a simplified constant-correlation covariance construction; this distinction is important for accurate interpretation.

8 Walk-Forward Backtesting

The backtest engine evaluates strategies in a leakage-aware walk-forward manner. The key principle is that each rebalance decision is made using information available up to that date; realized returns over the subsequent interval update portfolio value.

Algorithm 2 Leakage-aware walk-forward backtest (high-level)

Require: Start/end dates, rebalance schedule, capital V_0 , strategy π

- 1: Initialize portfolio value $V \leftarrow V_0$
 - 2: **for** each rebalance date d_t in the schedule **do**
 - 3: Build features from data up to d_t (no look-ahead)
 - 4: Obtain model predictions and compute allocation weights $w_t = \pi(\text{features}, \text{predictions})$
 - 5: Apply realized returns over $(d_t, d_{t+1}]$ to update V
 - 6: **end for**
 - 7: Compute summary metrics (CAGR, Sharpe, drawdown) and return time series
-

8.1 Example reproducible run (2023-01 to 2024-01)

The repository includes artifacts for a monthly-rebalanced walk-forward run from 2023-01-01 to 2024-01-01 with capital 100,000 and top- $N = 10$ selection. Table 5 reports the stored summary metrics.

Table 5: Backtest summary (monthly rebalancing, 2023-01-01 to 2024-01-01; stored artifact).

Metric	LINEAR strategy	Benchmark
CAGR (%)	48.31	30.60
Sharpe	3.043	1.752
Sortino	14.244	5.038
Max drawdown (%)	-1.165	-4.181
Final value	148271.84	130578.43

8.2 Backtest variants (stored summaries)

To probe sensitivity to rebalance cadence and evaluation window, the repository includes a variants summary artifact for the LINEAR strategy across multiple configurations. Table 6 reports these stored summaries. (Note: some risk metrics may appear as 0 or N/A for short horizons or small numbers of rebalance periods, reflecting the artifact values.)

Table 6: Stored backtest variants for LINEAR strategy (from repository variants summary artifact).

Variant	CAGR (%)	Sharpe	Volatility	Max DD (%)	Rebalances
Monthly (2023-01 to 2024-01)	48.31	3.043	11.71	-1.17	12
Quarterly (2023-01 to 2024-01)	40.54	3.725	27.97	0.00	4
Monthly (2022-01 to 2024-01)	37.47	1.796	15.80	-5.31	24

8.3 Visualization

Figures 1 and 2 show the stored equity curve and drawdown plots for the same run.

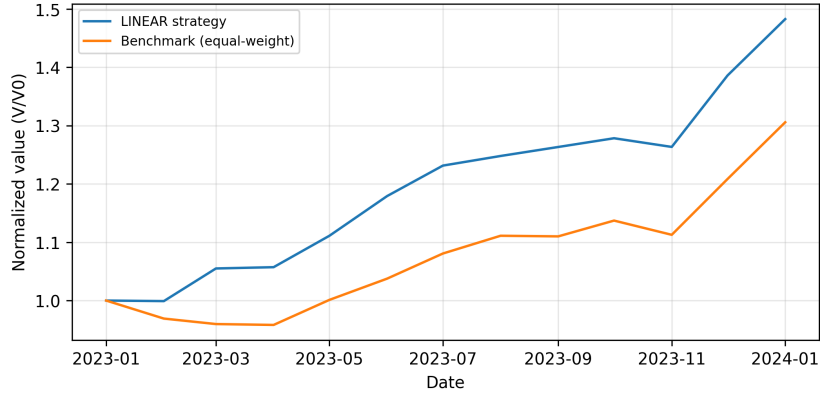


Fig. 1: Equity curve: LINEAR strategy vs benchmark (stored figure).

9 Frontend and User Interaction

QuantFin includes a React/TypeScript dashboard that calls backend endpoints to visualize prices, model outputs, allocations, and backtest results. The frontend is designed to provide:

- interactive selection of symbols and model families,
- inspection of training summaries and metrics,
- portfolio composition and health views,
- execution and visualization of walk-forward backtests.

The paper does not claim user-study outcomes; instead, it documents the implemented interaction surface.

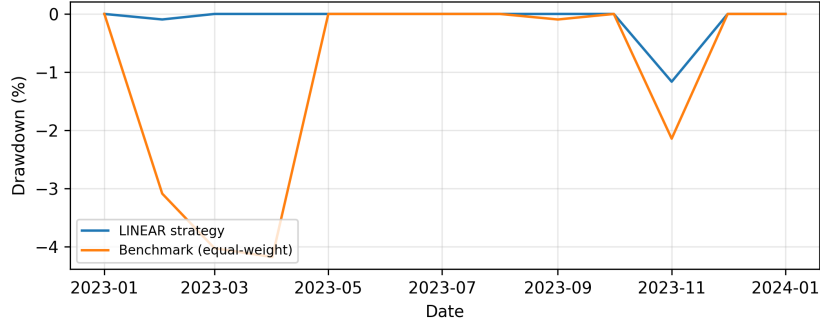


Fig. 2: Drawdown: LINEAR strategy vs benchmark (stored figure).

10 Engineering Validation Snapshot

Because QuantFin is a research prototype, it is important to explicitly document engineering status rather than imply production readiness. The repository includes a validation report artifact that executes a small suite of environment and integration checks. Table 7 summarizes the stored outcomes, and the report marks `production_ready` as `false`.

Table 7: Validation report snapshot (from repository validation artifact).

Check	Passed	Details (stored)
Environment Setup	Yes	Packages installed; 49 data files found
API Route Registration	No	attempted relative import beyond top-level package
Backtester Module	No	0 passed, 0 failed (suite did not run successfully)
Integration Tests	No	Backtester.__init__() missing required argument: <code>models_dir</code>

11 Limitations and Threats to Validity

This work is constrained by the repository’s scope and artifacts.

- **Transaction costs and slippage:** the stored backtest assumptions do not include explicit transaction costs or market impact; therefore returns may be optimistic relative to real trading.
- **Model comparability:** models solve different tasks (regression, classification, forecasting). Direct comparison requires careful alignment of targets and metrics.

- **Covariance estimation details:** some optimization paths use simplified covariance assumptions (notably in the online prediction service), while the backtest engine can estimate covariance from historical returns. Results should be interpreted with this distinction.
- **Engineering readiness:** repository validation artifacts indicate the system is not production-ready; operational robustness is outside this paper’s claims.

12 Conclusion and Future Work

This paper presented QuantFin, a full-stack, repository-implemented decision support system for equity investing that combines multi-header data ingestion, technical-indicator features, multiple ML baselines, and leakage-aware walk-forward backtesting. By grounding claims in saved artifacts, the paper provides a reproducible description of both methodology and observed outcomes.

Future work includes (i) richer multi-asset feature representations (e.g., explicit cross-asset structure), (ii) cost-aware backtesting with realistic frictions, (iii) better calibrated uncertainty estimates for ranking and allocation, and (iv) expanding stored evaluation artifacts to cover all implemented baselines (including ARIMA) across a broader symbol set.

Bibliography

- [1] Prakash K. Aithal, M. Geetha, Dinesh U, Basri Savitha, and Parthiv Menon. Real-time portfolio management system utilizing machine learning techniques. *IEEE Access*, 11:32595–32608, 2023.
- [2] Zihao Gao. Stock price prediction with arima and deep learning models. In *2021 IEEE 6th International Conference on Big Data Analytics (ICBDA)*, pages 61–68, 2021.
- [3] Abhishek Gunjan and Siddhartha Bhattacharyya. A brief review of portfolio optimization techniques. *Artificial Intelligence Review*, 56(5):3847–3886, 2022.
- [4] Vasilis Karlis, Katerina Lepenioti, Alexandros Bousdekis, and Gregoris Mentzas. Stock trend prediction by fusing prices and indices with lstm neural networks. In *2021 12th International Conference on Information, Intelligence, Systems & Applications (IISA)*, pages 1–7, 2021.
- [5] Qing Yang Eddy Lim, Qi Cao, and Chai Quek. Dynamic portfolio rebalancing through reinforcement learning. *Neural Computing and Applications*, 34(9):7125–7139, 2022.
- [6] Cong Ma, Jianshe Zhang, Zongxin Li, and Shuang Xu. Multi-agent deep reinforcement learning algorithm with trend consistency regularization for portfolio management. *Neural Computing and Applications*, 35(9):6589–6601, 2023.
- [7] Suman Saha, Junbin Gao, and Richard Gerlach. Stock ranking prediction using list-wise approach and node embedding technique. *IEEE Access*, 9:88981–88996, 2021.